

Spark-X2.5-4B on Jetson 8GB

Complete Setup Guide: llama.cpp + Spark Model + Hermes Agent Linking

This guide walks through compiling llama.cpp with Spark 2.5 architecture support, downloading the Spark-X2.5-4B Q4_K_M model, launching it at 32K context on a Jetson Orin Nano 8GB, and linking it to a Hermes Agent instance as a local custom provider.

Metric	Value
Model	Spark-X2.5-4B (4.1B params)
Quantization	Q4_K_M (2.60 GB)
Context window	32,768 tokens (32K)
KV cache	q8_0 (2.4 GB)
Generation speed	14.4 tok/s
BFCL v2 score	47/52 (90.4%)
Memory used	~5.0 GB of 7.4 GB
Architecture	spark2_5 (hybrid SWA + full attention)
Native context	1,048,576 tokens (1M)

Prerequisites

- NVIDIA Jetson Orin Nano 8GB (or equivalent aarch64 with 8GB unified memory)
- JetPack 5.2+ (L4T R36.x), CUDA 12.x, cuDNN installed
- Ubuntu 22.04 (aarch64) -- standard on JetPack
- At least 6 GB free memory (GUI/GDM should be stopped to free VRAM)
- Internet access for cloning llama.cpp and downloading the model
- Hermes Agent installed (or install via the instructions in Step 5)

Step 1: Free GPU Memory (Stop GUI)

On 8GB Jetson, the desktop GUI consumes ~1.5 GB of unified memory. Stop it before running any LLM server:

```
sudo systemctl stop gdm.service
```

```
# Verify: you should see ~6 GB available  
free -h
```

Note: To re-enable the GUI later, run: `sudo systemctl start gdm.service`

Step 2: Compile llama.cpp with CUDA + Spark 2.5 Support

Spark-X2.5 uses the spark2_5 architecture (hybrid sliding-window + full attention). You need a recent llama.cpp build that includes this architecture. As of October 2026, build 1051 (commit 8345f333) or newer is required.

2a. Clone and prepare:

```
cd ~
git clone https://github.com/ggerganov/llama.cpp.git
cd llama.cpp
```

2b. Configure with CUDA for Jetson (aarch64):

```
mkdir build && cd build

cmake .. \
  -DCMAKE_BUILD_TYPE=Release \
  -DGGML_CUDA=ON \
  -DCMAKE_CUDA_ARCHITECTURES=870 \
  -DGGML_CUDA_FA_ALL_QUANTS=ON
```

The CUDA architecture 870 targets the Jetson Orin GPU (Ampere). FA_ALL_QUANTS enables flash attention with all quantization types, which is essential for the q8_0 KV cache used in our optimal config.

2c. Build (takes ~15-25 minutes on Jetson):

```
cmake --build . --config Release -j 6

# Verify the build
./bin/llama-server --version
# Should print: version: 0.6.0-dev (build 1051 or higher)
```

Step 3: Download the Spark-X2.5-4B Model

Download the Q4_K_M quantization (2.60 GB) -- the tested and verified format for 8GB Jetson:

Option A: Hugging Face CLI (recommended):

```
pip install -U 'huggingface_hub[cli]'  
  
mkdir -p ~/models  
cd ~/models  
  
hf download stornic56/Spark-X2.5-4B-GGUF \  
  --include 'Spark-X2.5-4B-Q4_K_M.gguf' \  
  --local-dir .
```

Option B: Direct download with wget:

```
mkdir -p ~/models  
wget -O ~/models/Spark-X2.5-4B-Q4_K_M.gguf \  
  https://huggingface.co/stornic56/Spark-X2.5-4B-GGUF/resolve/main/Spark-X2.5-4B-Q4_K_M.gguf
```

Verify the download:

```
# Check file size (~2.6 GB)  
ls -lh ~/models/Spark-X2.5-4B-Q4_K_M.gguf  
  
# Verify GGUF architecture by loading metadata only  
~/llama.cpp/build/bin/llama-server \  
  -m ~/models/Spark-X2.5-4B-Q4_K_M.gguf \  
  --host 127.0.0.1 --port 19200 \  
  -ngl 0 -c 512 -lv 4 2>&1 | head -20  
  
# Look for: general.architecture = spark2_5  
# spark2_5.block_count = 36  
# spark2_5.context_length = 1048576
```

Step 4: Launch the Server with Optimal 32K Settings

These settings were empirically verified on Jetson Orin Nano 8GB to achieve 14.4 tok/s at 32K context with q8_0 KV cache -- the same speed as 16K, with double the context window.

Key environment variables (both required):

Variable	Purpose
GGML_CUDA_ENABLE_UNIFIED_MEMORY=1	Allows GPU to use system RAM via NvMap (essential on Jetson unified memory)
GGML_CUDA_NO_PINNED=1	Disables CUDA pinned memory, saving ~500 MB (critical for 32K on 8GB)

Launch command:

```
GGML_CUDA_ENABLE_UNIFIED_MEMORY=1 \
GGML_CUDA_NO_PINNED=1 \
~/llama.cpp/build/bin/llama-server \
-m ~/models/Spark-X2.5-4B-Q4_K_M.gguf \
--alias spark-x2.5-4b \
--host 127.0.0.1 --port 8090 \
-ngl 99 \
-c 32768 \
-ctk q8_0 -ctv q8_0 \
-b 512 -ub 512 \
-fa on --jinja -np 1 -t 6 \
--temp 0.0 --top-k 1 --repeat-penalty 1.0 \
--fit off
```

Flag explanation:

Flag	Value	Why
-ngl	99	Full GPU offload -- all 36 layers on GPU for max speed
-c	32768	32K context window (2x the safe 16K default)
-ctk / -ctv	q8_0	8-bit KV cache -- better quality than q4_0, avoids fragmentation OOM
-b / -ub	512	Batch size for prompt processing throughput
-fa	on	Flash attention -- reduces memory and speeds up long context
--jinja		Enables OpenAI-compatible tool/function calling (required for Hermes)
-np	1	Single slot -- no parallel requests (saves memory on 8GB)
-t	6	Use 6 CPU threads (Jetson has 6 cores)
--temp	0.0	Greedy decoding for deterministic tool calling
--top-k	1	Greedy -- pick top token only
--fit off		Disable context auto-shrinking (we want full 32K reserved)

Verify the server is running:

```
# Wait ~25 seconds for model to load, then check health
curl -s http://127.0.0.1:8090/health
# Should return: {"status":"ok"}

# Quick smoke test
curl -s http://127.0.0.1:8090/v1/chat/completions \
-H 'Content-Type: application/json' \
-d '{"model":"spark-x2.5-4b","messages":[{"role":"user","content":"Say
```

```
hello"}], "max_tokens": 50}'
```

Why q8_0 KV Cache (Not q4_0)?

Counterintuitively, q8_0 KV cache works better than q4_0 at 32K on 8GB Jetson. While q8_0 uses more memory (2.4 GB vs 1.2 GB for the KV buffer), it pre-allocates the full buffer upfront in a way the Linux kernel handles more predictably. The q4_0 variant, despite being smaller, triggers OOM from memory fragmentation during generation -- it loads and passes the first test, then crashes on the second.

Result: q8_0 gives you 32K context at 14.4 tok/s with zero crashes. q4_0 at 32K requires the -nkvo flag (KV on CPU), which drops speed to 10.5 tok/s (-30%).

Step 5: Link to Hermes Agent

The llama-server exposes an OpenAI-compatible API at <http://127.0.0.1:8090/v1>. Hermes Agent's custom provider connects directly -- no API key needed for local models.

5a. Install Hermes Agent (if not already installed):

```
curl -fsSL https://hermes-agent.nousresearch.com/install.sh | bash
```

```
# Or if already installed, just verify:
hermes --version
```

5b. Configure Hermes to use Spark:

```
hermes config set model.default spark-x2.5-4b
hermes config set model.provider custom
hermes config set model.base_url http://127.0.0.1:8090/v1
hermes config set model.api_key none
hermes config set model.context_length 32768
```

5c. Verify the connection:

```
# Start Hermes in chat mode
hermes chat

# Type a message -- it should route to your local Spark server.
# The llama-server terminal will show the request being processed.
```

5d. Configure tool calling (optional but recommended):

Spark-X2.5-4B scored 90.4% on BFCL v2 function-calling benchmarks. To enable tool use in Hermes with this model, ensure the server was launched with --jinja (it was, in Step 4). Hermes will automatically send tool definitions and parse the model's function-call responses.

For best tool-calling accuracy, use greedy decoding (temp 0.0, top_k 1) which is already set in the server launch command. For creative/chat tasks, you can override per-request with higher temperature.

Appendix A: Launch Script

Save this as `~/launch_spark.sh` for one-command startup:

```
#!/bin/bash
# Spark-X2.5-4B optimal settings for Jetson Orin Nano 8GB
# 32K context, q8_0 KV, 14.4 tok/s, BFCL 90.4%
# Usage: bash launch_spark.sh [port] (default 8090)

PORT=${1:-8090}

GGML_CUDA_ENABLE_UNIFIED_MEMORY=1 \
GGML_CUDA_NO_PINNED=1 \
~/llama.cpp/build/bin/llama-server \
-m ~/models/Spark-X2.5-4B-Q4_K_M.gguf \
--alias spark-x2.5-4b \
--host 127.0.0.1 --port $PORT \
-ngl 99 -c 32768 \
-ctk q8_0 -ctv q8_0 \
-b 512 -ub 512 \
-fa on --jinja -np 1 -t 6 \
--temp 0.0 --top-k 1 --repeat-penalty 1.0 \
--fit off
```

Appendix B: Spark-X2.5-4B Architecture Details

Parameter	Value
Architecture	spark2_5 (hybrid SWA + full attention, 3:1 ratio)
Parameters	4.1B
Layers (block_count)	36
Embedding dimension	2560
Feed-forward length	10240
Attention heads	16
KV heads (GQA)	4
Key/Value length	256
RoPE base	5,000,000 (full), 10,000 (SWA)
Native context	1,048,576 (1M tokens)
Vocab size	~152K
File size (Q4_K_M)	2.60 GB (2,600,224,352 bytes)
Tensors	290

Appendix C: Troubleshooting

Server fails to start / OOM kill:

- Stop GDM: `sudo systemctl stop gdm.service`
- Check free memory: `free -h` (need 6+ GB available)
- Kill stale processes: `kill -9 -f llama-server`

- If still OOMing at 32K, fall back to 16K: change -c 32768 to -c 16384
- Last resort -- KV on CPU (slower): add -nkvo -ub 256 -b 256 flags (drops to ~10.5 tok/s)

Model loads but tool calls fail:

- Ensure --jinja is in the launch command (enables chat template + tool format)
- Ensure --temp 0.0 --top-k 1 for greedy decoding (best for tool calling)
- Verify Hermes config: hermes config get model.provider should show 'custom'
- Check the server logs for parse errors on tool-call responses

'spark2_5 architecture not found':

Your llama.cpp build is too old. Spark 2.5 support was added in October 2026. Update to the latest commit and rebuild:

```
cd ~/llama.cpp && git pull
cd build && cmake --build . --config Release -j 6
```

Connection refused on port 8090:

- Wait 25-30 seconds for the model to fully load before connecting
- Check: curl http://127.0.0.1:8090/health
- Check: pgrep -c llama-server (should be 1)
- Try a different port: bash launch_spark.sh 8091

Verified October 2026 on Jetson Orin Nano 8GB (L4T R36.5.2, JetPack 5.2, llama.cpp build 1051, commit 8345f333).
BFCL v2 scores from the 32K Context LLM Survey project. Full results and launch scripts at the project repository.